Project Title: *Pushing the Limits of what a Transformer can do – TinyStories*

Abstract

This project critically investigates the capabilities of small language models (SLMs) on the TinyStories dataset. While the original work highlighted the success of the Transformer, we question its architectural necessity and examine the dataset's underlying quality. We design and benchmark four models: a standard Transformer baseline, a surprisingly effective Linear Autoregressive Model (MLP), a novel Contrastive Transformer for improved coherence, and a memory-efficient Compressor model. Our dual evaluation framework, combining LLM-based judgments with objective metrics like BLEU and ROUGE, reveals significant data quality issues in TinyStories. Our central finding is that **even simple autoregressive models can achieve remarkable narrative fluency, suggesting the learning paradigm itself, not architectural complexity, is the primary driver of performance** on this simplified domain. This study offers a richer perspective on designing lightweight, effective models tailored to specialized tasks.

1. Introduction & Motivation

1.1 The Challenge of Large Language Models (LLMs)

Most state-of-the-art language models, such as GPT-3 and its successors, are extremely large and computationally demanding. Their high resource requirements limit their accessibility and practical application in real-time or resource-constrained environments like mobile devices or educational tools. This size and complexity also introduce substantial barriers to understanding and controlling their behavior, a critical aspect for specialized tasks like narrative generation.

1.2 The "TinyStories" Proposition

The paper TinyStories: How Small Can Language Models Be and Still Speak Coherent English? by Eldan and Li (2023) introduced a compelling counter-narrative. They proposed that the primary obstacle for smaller models isn't their parameter count, but the overwhelming complexity of typical training data.

They introduced TinyStories, a synthetic dataset of short stories using language understandable by a 3- to 4-year-old. Their work demonstrated that models with fewer than 10 million parameters, when trained on this simplified dataset, could produce coherent, grammatically near-perfect stories, significantly outperforming much larger models trained on unfiltered web data.

The TinyStories dataset consists of 2.1 million stories for training and 22k for validation, all generated by GPT-4. The dataset is designed to be simple, ethical, and free from offensive material, making it ideal for educational and creative AI applications. They also explore how models much smaller than SOTA produce coherent, consistent and diverse stories with near-perfect grammar. They employ GPT-4 eval, proposing that it overcomes flaws of traditional benchmarks that demand structure and are not multidimensional. The authors hope this can facilitate research into LMs for low-resource or highly specialised domains.

1.3 Our Research Questions

Inspired by this work, our project undertakes a multi-faceted investigation to push these limits further and critically analyze the paper's claims. Rather than simply reproducing results, we sought to deconstruct the problem by answering several key questions:

1. **Dataset Robustness:** How reliable is the TinyStories dataset itself, and what impact do its characteristics have on model training?
2. **Architectural Necessity:** Is the full Transformer architecture truly necessary for this task, or can simpler models suffice?
3. **Methodological Innovation:** Can novel training methods or architectures improve performance or efficiency over the baseline?
4. **Evaluation Reliability:** How do traditional, objective metrics compare to the paper's subjective GPT-eval, and what are the limitations of each?

2. The TinyStories Dataset: The Good, the Bad, and the Ugly

2.1 The Intended Design

TinyStories is a synthetic dataset created using GPT-3.5 and GPT-4. It contains short stories (2–3 paragraphs) with simple sentences, consistent themes, and vocabulary suitable for 3- to 4-year-old children. The dataset was designed to capture key aspects of natural language—grammar, reasoning, and narrative structure—while limiting complexity.

An example from the dataset:

Once upon a time, in an ancient house, there lived a girl named Lily. She loved to decorate her room with pretty things. One day, she found a big box in the attic. She opened it and saw many shiny decorations. Lily was very happy and decided to use them in her room.

As Lily was decorating her room, the sky outside became dark. There was a loud thunder sound, and Lily got scared. She ran to her mom and said, "Mommy, the thunder is so loud!" Her mom hugged her and said, "Don't worry, it will pass soon." But the thunder did not stop. It got louder and louder, and the ancient house started to shake. Suddenly, the roof fell down on the pretty decorations. Lily was sad because her room was not pretty anymore. The end.

The TinyStories dataset is found on Hugging Face.

2.2 A Critical Analysis of Data Quality

While the paper's premise is strong, a closer inspection of the dataset revealed several significant quality issues not addressed by the original authors.

The dataset, despite its scale of over 200,000 samples, exhibits significant quality issues stemming from inadequate pre-processing and oversight.

Pre-processing and Cleaning Deficiencies

- **Spurious Characters:** The dataset contains irrelevant elements such as CJK ideographs, emojis, and improper ASCII quotations, hyphens, and ellipses. For instance, ASCII quotations appear in 10,000 samples, while them-dashes are present in 3,000.

- **Incomplete or Malformed Stories:** Some stories are truncated or lack logical conclusions. Extreme cases include blank entries or non-narrative content, such as a story that devolves into counting numbers from 1 to 194.

Vocabulary Mismatch

The authors claim the dataset is built on a vocabulary of ~1,500 basic words. However, tokenizer training reveals a vocabulary size closer to 50,000 words (using BPE encoding), including complex terms like "conglomerate," which are inappropriate for the intended audience, such as children.

Reproducibility and Evaluation Concerns

The paper lacks clear instructions for reproducing the results.

The GPT-eval metric is ambiguously defined, making it difficult to validate the authors' claims.

Data Contamination

The dataset contains artifacts like empty stories, duplicates, and non-narrative content, further compromising its integrity.

These issues suggest a lack of rigorous preprocessing, which can impact the training of smaller models that are more sensitive to data quality.

```
Lily and Ben were friends. They liked to play with toys and run in the park. One day, they found a big cake on the table. It
looked yummy and sweet. They wanted to eat some. But Mom said, "No, no, no. That cake is for Grandma's birthday. You can't have
any. It is not for you." Lily and Ben were sad and angry. They did not like Mom's words. They did not want to wait for Grandma.
They wanted cake now. They had a bad idea. They decided to sneak some cake when Mom was not looking. They took a big knife and
cut a slice. They put it on a plate and ran to the corner. They took a bite of the cake. But it was not yummy and sweet. It was
disgusting and bitter. It had salt and pepper and vinegar and mustard and garlic and onion and cheese and fish and pickle and
soap and dirt and bugs and worms and hair and nails and glass and metal and rocks and sticks and bones and blood and poop and pee
and spit and snot and pus and vomit and slime and goo and mold and rot and rust and dust and ash and trash and fire and ice and
pain and hate and death and doom and gloom and fear and tears and screams and nightmares and curses and sins and hell and evil
and darkness and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and
nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing
and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and
nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing
and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and
nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing
and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and
nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing
and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and
nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing
and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and
nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing
and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and nothing and
```

```
open it and see many shiny things. They are coins! Sam and Tom are very happy. They have never seen so many coins before. "Wow,
we are so lucky!" Sam says. "Yes, we are! How many coins do you think there are?" Tom says. They start to count the coins. But
they have a problem. They do not know how to count very well. They only know how to count to ten. After ten, they get confused.
"Ten, eleven, twelve, thirteen, fourteen, fifteen, sixteen, seventeen, eighteen, nineteen, twenty, twenty-one, twenty-two,
twenty-three, twenty-four, twenty-five, twenty-six, twenty-seven, twenty-eight, twenty-nine, thirty, thirty-one, thirty-two,
thirty-three, thirty-four, thirty-five, thirty-six, thirty-seven, thirty-eight, thirty-nine, forty, forty-one, forty-two, forty-
three, forty-four, forty-five, forty-six, forty-seven, forty-eight, forty-nine, fifty, fifty-one, fifty-two, fifty-three, fifty-
four, fifty-five, fifty-six, fifty-seven, fifty-eight, fifty-nine, sixty, sixty-one, sixty-two, sixty-three, sixty-four, sixty-
five, sixty-six, sixty-seven, sixty-eight, sixty-nine, seventy, seventy-one, seventy-two, seventy-three, seventy-four, seventy-
five, seventy-six, seventy-seven, seventy-eight, seventy-nine, eighty, eighty-one, eighty-two, eighty-three, eighty-four, eighty-
five, eighty-six, eighty-seven, eighty-eight, eighty-nine, ninety, ninety-one, ninety-two, ninety-three, ninety-four, ninety-
five, ninety-six, ninety-seven, ninety-eight, ninety-nine, one hundred, one hundred and one, one hundred and two, one hundred and
three, one hundred and four, one hundred and five, one hundred and six, one hundred and seven, one hundred and eight, one hundred
and nine, one hundred and ten, one hundred and eleven, one hundred and twelve, one hundred and thirteen, one hundred and
fourteen, one hundred and fifteen, one hundred and sixteen, one hundred and seventeen, one hundred and eighteen, one hundred and
nineteen, one hundred and twenty, one hundred and twenty-one, one hundred and twenty-two, one hundred and twenty-three, one
hundred and twenty-four, one hundred and twenty-five, one hundred and twenty-six, one hundred and twenty-seven, one hundred and
twenty-eight, one hundred and twenty-nine, one hundred and thirty, one hundred and thirty-one, one hundred and thirty-two, one
hundred and thirty-three, one hundred and thirty-four, one hundred and thirty-five, one hundred and thirty-six, one hundred and
thirty-seven, one hundred and thirty-eight, one hundred and thirty-nine, one hundred and forty, one hundred and forty-one, one
hundred and forty-two, one hundred and forty-three, one hundred and forty-four, one hundred and forty-five, one hundred and
forty-six, one hundred and forty-seven, one hundred and forty-eight, one hundred and forty-nine, one hundred and fifty, one
hundred and fifty-one, one hundred and fifty-two, one hundred and fifty-three, one hundred and fifty-four, one hundred and fifty-
```

Figure 1: Examples of problematic data found in the TinyStories dataset, including repetitive, non-narrative, and nonsensical entries.

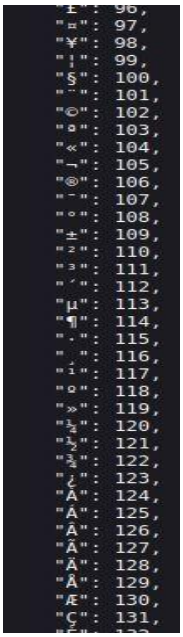


Figure 2: A snapshot from the tokenizer vocabulary, showing its large size (50k+) and inclusion of complex or irrelevant characters

3. Methodology

To answer our research questions, we implemented, trained, and compared four distinct model architectures.

3.1 The Baseline: A Decoder-Only Transformer

To establish a performance baseline, we implemented a standard decoder-only Transformer from scratch using PyTorch's nn.TransformerEncoderLayer. This model adheres to the original architecture described in the foundational paper and is specifically designed for autoregressive tasks, generating output one token at a time, with each step dependent on the previously generated tokens. This design is ideal for sequential prediction tasks such as text generation or language translation.

3.2 Key Components of the BasicTransformer Model

Embedding Layer, the embedding layer converts each input token (e.g., a word or symbol) into a high-dimensional vector. These embeddings are scaled by the square root of the model dimension d_{model}, ensuring appropriate value scaling before further processing. This layer is critical, as it provides the foundational numerical representation of tokens, enabling the model to capture semantic and syntactic information beyond raw token indices.

Positional Encoding Since the transformer architecture does not inherently capture token order, positional encoding is added to the token embeddings. This encoding injects information about the position of each token in the sequence, allowing the model to differentiate between token positions and generate sequences in the correct order. Positional encoding is essential for maintaining the contextual integrity of the input.

Self-Attention Mechanism The multi-head self-attention mechanism is a core feature of the model. It allows each token in the input sequence to "attend" to other tokens, enabling the model to focus on different parts of the sequence when making predictions. In autoregressive models, self-attention is masked so that each token only attends to preceding tokens, ensuring step-by-step generation without access to future information. This mechanism is vital for understanding long-range dependencies and contextual relationships between tokens.

Feed-Forward Network (FFN) After self-attention, the output is processed through a feed-forward network (FFN), which consists of two linear transformations separated by a non-linear activation function (e.g., ReLU). The FFN refines token representations, enhancing the model's ability to generate coherent and contextually meaningful sequences.

Layer Normalization Layer normalization is applied before the self-attention and feed-forward layers. This stabilizes training by normalizing the outputs of these layers, keeping values within a suitable range and reducing the risk of training instability.

Final Projection Layer The processed sequence is passed through a final projection layer, which converts the internal token representations into logits—probability distributions over the vocabulary. These logits determine the likelihood of each possible next token, guiding the autoregressive generation process.

Autoregressive Nature The model's autoregressive design ensures that tokens are generated sequentially, with each token dependent solely on those that precede it. This forward-only generation process is fundamental for tasks like language modeling, where predicting the next token based on previous context is central to the task.

This implementation serves as a robust baseline for evaluating performance in autoregressive text generation.

3.3 Model Parameters Calculation

In a decoder-only transformer model, the total number of parameters can be divided into several key components:

Embedding Layer: The embedding layer maps input tokens into high-dimensional vectors. The number of parameters is:

$$\text{Embedding\_params} = \text{vocab\_size} \times d_{\text{model}}$$

Multi-Head Self-Attention (MHA): The self-attention mechanism consists of several learned matrices for query, key, value, and output projections. The total number of parameters is:

$$\text{MHA\_params} = 4 \times d^2_{\text{model}} + 4 \times d_{\text{model}}$$

Feed-Forward Neural Network (FFNN): Each layer contains a two-layer feed-forward net- work. The number of parameters is:

$$\text{FFNN\_params} = 2 \times d_{\text{model}} \times \text{dim\_feedforward} + d_{\text{model}} + \text{dim\_feedforward}$$

Layer Normalization: Layer normalization is applied twice in each layer, once before the self- attention mechanism and once before the feed-forward network. The number of parameters is:

$$\text{LayerNorm\_params} = 2 \times 2 \times d_{\text{model}}$$

Final Feed-Forward Layer: The final feed-forward layer projects the model's output from

dmodel dimensions to the vocabulary size to generate logits for each token. The parameters are:

$$\text{Final\_FFN\_params} = d_{\text{model}} \times \text{vocab\_size} + \text{vocab\_size}$$

Total Parameters:

$$\begin{aligned}\text{Total\_params} = {} & \text{Embedding\_params} \\ & + \text{num\_layers} \times (\text{MHA\_params} + \text{FFNN\_params} + \text{LayerNorm\_params}) \\ & + 2 \times \text{LayerNorm\_params} \\ & + \text{Final\_FFN\_params}\end{aligned}$$

3.4 The Challenger: A Linear Autoregressive Model (MLP)

To question the necessity of the Transformer's complexity, we implemented a much simpler **Linear Model**, a **Multi-Layer Perceptron (MLP)**, inspired by the paper *"Auto-Regressive Next-Token Predictors are Universal Learners"* (Malach, 2023). This model replaces the self-attention mechanism with **masked linear layers**, preserving the **autoregressive property**—predicting the next token based only on previous ones—while avoiding the quadratic complexity of attention. Its surprising effectiveness on the **TinyStories dataset** underscores the power of autoregressive learning itself.

3.4.1 Key Insights from the Study

1. The Power of Autoregression Autoregression is a deceptively simple yet powerful mechanism for modeling nonlinear patterns in sequential data. Malach's work demonstrates that autoregressive models can approximate any function computable by a Turing Machine. By iteratively updating inputs and labels with sampled information, even basic models can capture complex dependencies. This

process resembles a **chain-of-thought reasoning**, where each prediction builds on the previous context, enabling the model to generate coherent and structured outputs.

2. Simplicity of the Dataset The **TinyStories dataset** consists of simple, structured stories with clear language patterns. This simplicity allows even uncomplicated models—like MLPs and small transformers—to generate cohesive sentences. While the dataset lacks overly complex syntax, it retains enough structure to facilitate next-word prediction without requiring advanced logical reasoning. This suggests that **data structure and autoregressive training** often matter more than architectural complexity.

3. Why the Linear Model Works Despite its simplicity, the **MLP with 775M+ parameters** performs remarkably well on TinyStories. Its effectiveness stems from the dataset's inherent structure rather than architectural sophistication. The linear model's lack of activation functions and complex layers allows it to focus solely on the core task: **predicting the next word based on prior context**. In constrained domains like TinyStories, simpler architectures can achieve impressive results without the computational overhead of larger models.

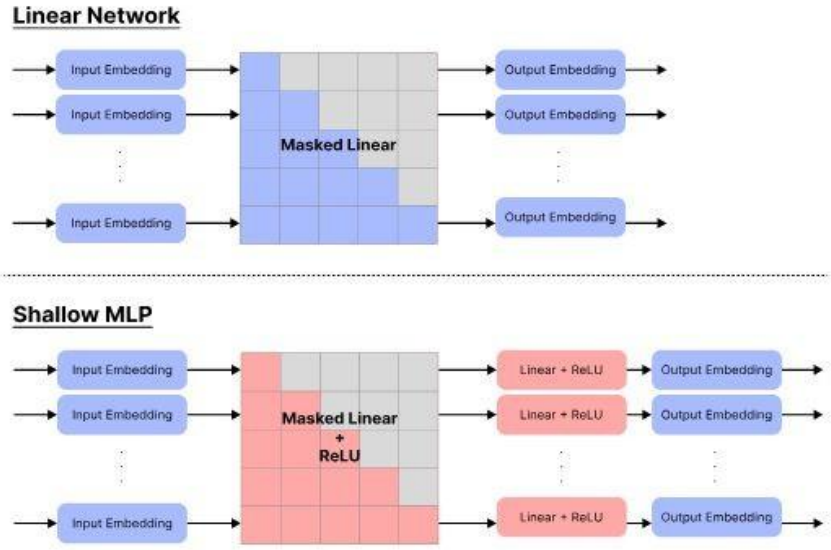
4. Why Small Transformers Still Excel Small transformers, though more complex than linear models, also thrive on this task. Their **multi-head self-attention mechanism** enables them to capture rich contextual relationships between words, enhancing coherence in generated text. Like MLPs, they benefit from the **autoregressive training process**, refining their predictions with each step. This demonstrates that even modestly sized transformers can deliver high-quality results for structured tasks without excessive scaling.

This analysis reveals that **autoregressive learning** is often the driving force behind model performance, rather than architectural intricacy. Both simple MLPs and small transformers excel on the TinyStories dataset, highlighting that **data structure and training methodology** can outweigh the need for complex architectures. The findings align with Malach's assertion: **autoregression alone can approximate universal learning**, making it a cornerstone of modern sequential modeling.

Example of a sentence generated from a linear model:

Once ufion a time, there was a little girl named Lily. She loved to filay outside in the fiark. One day, she saw a big tree with lots of leaves. She wanted to climb it, but she couldn't climb it.

Suddenly, she heard a voice from behind her…..



Prompt #1: He was very scared

Output: *He was very scared. He cried out loud, but then he heard a voice. He was afraid of the dark monster. He ran to the bear. He had a pain in his hand. He screamed and ran away. He was never seen again. He was scared of the monster and he went on. He never...*

Prompt #2: Adam was hungry, so

Output: *Adam was hungry, so he wanted to eat the dessert. But he knew it was not a good at his sister . He thought, "Maybe I can eat this food, but I don't want to share with you."*

Prompt #3: Alice was tired, so

Output #1: *Alice was tired, so she decided to take a nap . She put a blanket on the ground and started to cry. Then, she heard a noise. It was a nearby tree.*

Output #2: *Alice was tired, so she decided to go on an adventure . She hopped on the way to go home and look for her...*

MODEL	GRAMMAR GPT-4 / LT	CREATIVITY GPT-4	CONSISTENCY GPT-4	PLOT GPT-4
TS-33M	8.0 ± 0.8 / 62%	7.2 ± 0.5	7.0 ± 1.2	6.9 ± 0.8
TS-1M	6.9 ± 0.9 / 59%	6.7 ± 1.0	6.0 ± 1.5	5.6 ± 1.3
LINEAR	6.3 ± 2.0 / 64%	6.2 ± 1.8	5.9 ± 1.8	5.2 ± 1.8

Figure 2. **Top**: Example prompts and outputs for Linear model trained on TinyStories (grammatical/conceptual errors in red). **Bottom**: Comparison between Transformer-and Linear models, average grades from GPT-4.

Figure 3: (First image) The simple architecture of our Linear and Shallow MLP models. (Second image) Example outputs showing surprisingly coherent, though sometimes grammatically flawed, generation.

3.5 The Innovator: A Contrastive Learning Transformer

To improve narrative coherence, we developed a **Contrastive Transformer**. This model uses the same architecture as our baseline but introduces a novel loss function. During each training step, a subset of stories has its sentences shuffled. The model is then trained to maximize the standard cross-entropy

loss on correct stories (Ln) while simultaneously maximizing the loss on the incoherent, shuffled stories (Lc), guided by the formula: L = Ln + α / (Lc + 0.01). This explicitly teaches the model to recognize and generate logical plot progression.

3.6 The Optimizer: A Memory-Efficient Compressor Model

To address the computational cost of self-attention, we designed the **Compressor Model**. This architecture consists of three stages:

1. **Encoder:** Compresses pairs of input tokens into single, richer vector representations.
2. **Trunk:** A standard Transformer that processes the sequence of compressed vectors, which is now half the original length. This dramatically reduces the memory cost of the attention mechanism.
3. **Decoder:** Decompresses the output from the trunk back into pairs of tokens for generation.

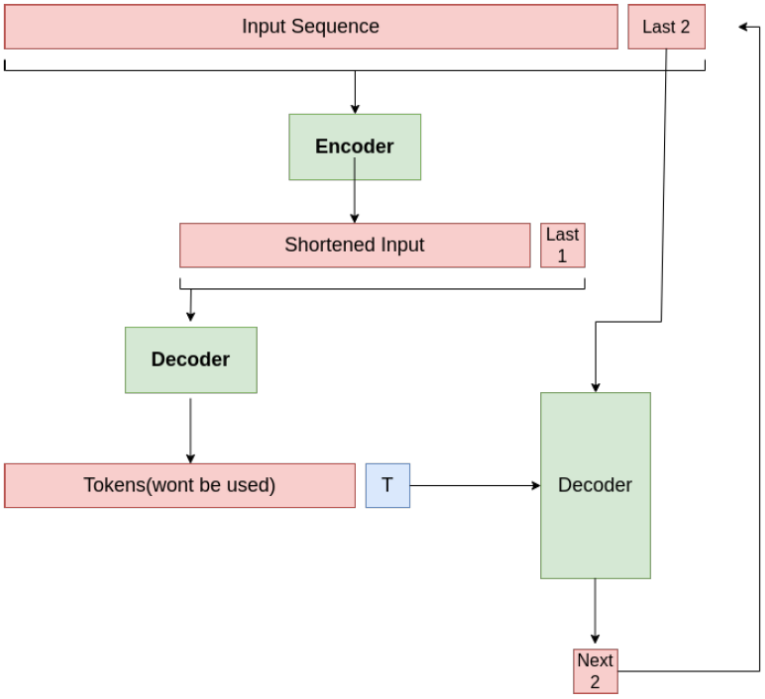


Figure 4: The architecture of our memory-efficient Compressor model, designed to reduce the sequence length processed by the main Transformer trunk.

4. Experimental Setup & Evaluation Framework

4.1 Training Ablations

We conducted several training runs to explore the impact of hyperparameters, including:

- Varying model dimensions (256, 512, 768, 1024).
- Varying layer counts (6, 12) and attention heads (2, 4, 8).
- Comparing training on 10% of the data for 10 epochs vs. 100% of the data for 1-2 epochs.

4.2 A Dual-Pronged Evaluation Strategy

A core part of our project was to critically assess evaluation itself. We employed both the paper's subjective method and a suite of objective, traditional metrics to ensure a comprehensive analysis.

4.2.1 Subjective Evaluation

Following the paper's approach, we used **LLMs (GPT-3.5-Turbo and Gemini-1.5-Flash)** as judges. Generated stories were scored from 1 to 10 across four dimensions:

- Grammar
- Consistency
- Creativity
- Plot Sense

This subjective evaluation provides a nuanced understanding of the quality of generated text, capturing aspects that automated metrics might miss.

4.2.2 Objective Evaluation

To provide an objective and reproducible counterpoint, we evaluated all models using a standard suite of **machine translation and text similarity metrics**:

• **BERTScore** BERTScore measures the semantic similarity between predicted completions and reference texts using **contextual embeddings from BERT**. It computes precision, recall, and F1-score based on the cosine similarity between token embeddings, capturing deep, context-aware representations rather than surface-level matches. BERTScore is available in two variants: **raw (without rescaling)** and **rescaled (based on a baseline model)**.

• **BLEU (Bilingual Evaluation Understudy)** BLEU is a **precision-based metric** that evaluates the overlap of n-grams (e.g., unigrams, bigrams) between generated and reference texts. It produces a score between 0 and 1, with higher values indicating better quality. BLEU emphasizes precision and penalizes overly short or repetitive output, making it widely used in **machine translation**.

• **ROUGE (Recall-Oriented Understudy for Gisting Evaluation)** ROUGE measures **recall**, focusing on how much of the reference text is captured by the generated text. It includes several variants:

- **ROUGE-1** (unigrams)
- **ROUGE-2** (bigrams)
- **ROUGE-L** (longest common subsequence)

ROUGE is particularly useful for **summarization tasks**, where capturing the essence of the source text is critical.

• **ChrF (Character F-score)** ChrF is a **character-level evaluation metric** that calculates the F-score (harmonic mean of precision and recall) for character n-grams (e.g., bigrams or trigrams). It is effective for languages with **complex morphology** or unclear word boundaries, as it focuses on character-level precision and recall. The **beta parameter** allows adjustment of the recall-to-precision weighting,

making it versatile for tasks like **machine translation** or **text generation**, where minor spelling errors can impact word-level metrics.

This combination of subjective and objective metrics ensures a **balanced and rigorous evaluation**, capturing both the qualitative and quantitative aspects of model performance.

4.2.3 Contrast to the Original Paper

The original paper relies on GPT-eval, which uses the GPT-4 model to grade sentences out of 10 on Grammar, Creativity and Consistency.

Some of the concerns with this is that these are not objective or frozen. A sentence once evaluated a certain score may obtain a different score a few days later, bringing into question how reproducible and robust the evaluations of these models are by using GPT-eval. (Also we don't have money for GPT API Access :))

If some more objective method was available for comparing models, it would be a good addition to the GPT-eval method. In addition, these scoring methods are much more interpretable, so one can comment on what exactly certain models are doing right or wrong. For example, a high ChrF score but a low BERTScore could be indicative that the model is making typos (not likely, but imagine it as an example), or using similar-looking words that mean different thing. For example, the sentences "I'm indifferent." and "I'm different." would hypothetically get a high ChrF but low BERTScore.

4.2.4 Limitations of MT-focused Scores

As is clear from the name, they are used largely for machine translation. Standards are different when it comes to completing stories (the goal of our models) and machine translation. The latter demands a less flexible set of predictions from the models as compared to generation, which is the task at hand. Making judgments about creativity is not effective, and may actually be inversely proportional to these scoring methods.

4.2.5 Between Model Comparison

We computed the above metrics and scaled them to a 10-point range. Following are the hyperparameter

settings:

- Run 1: hidden dimension 512 | layers 6 | heads 8 | data 10% | epochs 10
- Run 2: hidden dimension 1024 | layers 6 | heads 8 | data 10% | epochs 10
- Run 3: hidden dimension 256 | layers 6 | heads 8 | data 10% | epochs 10
- Run 4: hidden dimension 512 | layers 6 | heads 8 | data 100% | epochs 1
- Run 5: hidden dimension 768 | layers 12 | heads 8 | data 100% | epochs 2

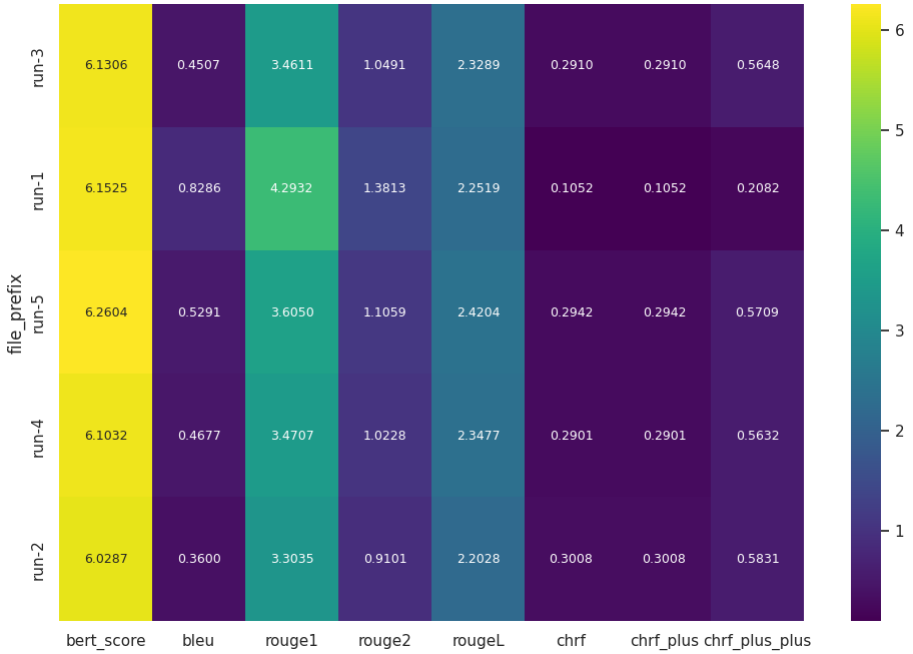


Figure 5: Comparison between All Models Trained

How Much of a Role does Number of Training Epochs Play?

We compare 2 models that have the same architecture and hyperparameter settings, except for the size of data and number of epochs. For one model, we used 10% of the data and ran for 10 epochs, and the other with 100% of the data for 1 epoch.

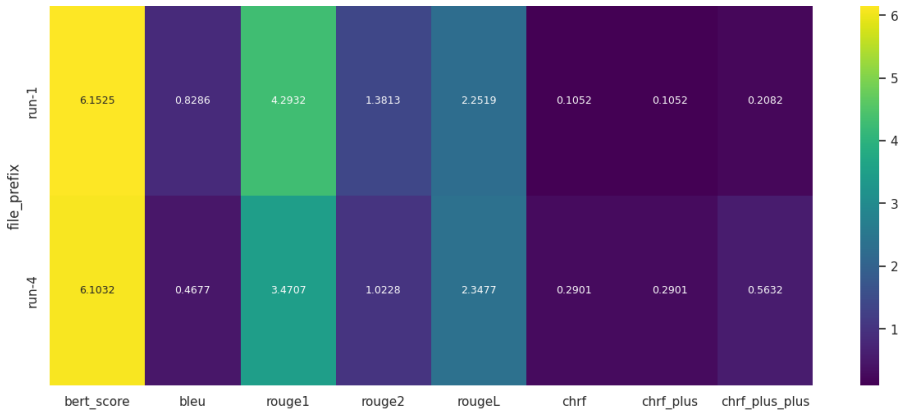


Figure 6: Comparison between Models Trained for Different Number of Epochs

We also compared the performance of one model over each of the 10 epochs it was trained for, by using model checkpoints.

Figure 7: Comparison between Epochs of the Same Model

Comparing the five models with the MLP, we can observe that the MLP performs worse compared to the other transformer models by these measures.

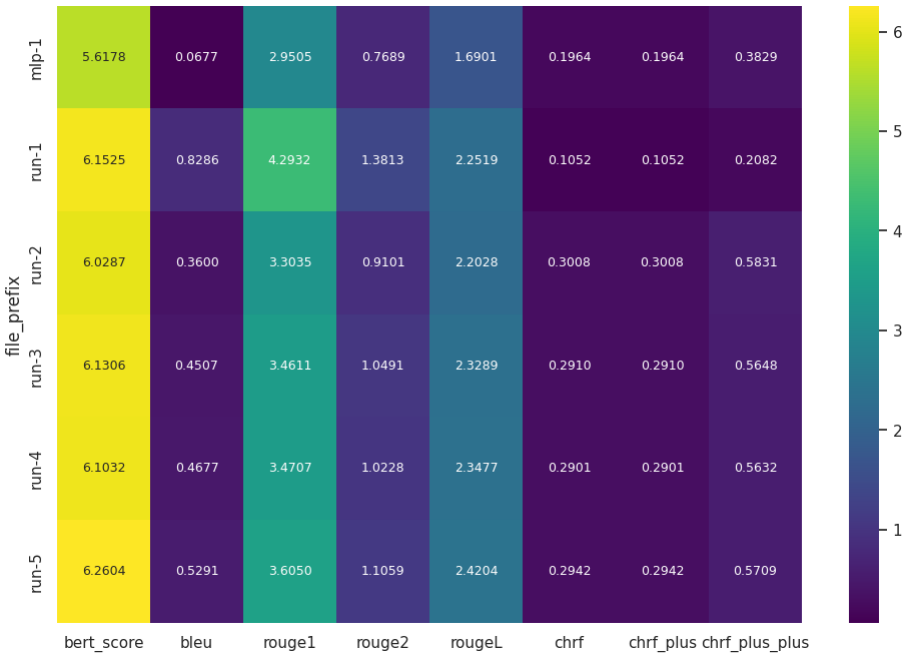


Figure 8: Comparison between MLP and the Transformers

5. Results and Comparative Analysis

5.1 Model Training and Convergence

All models successfully learned from the dataset, as evidenced by decreasing training and validation loss curves. A key finding was that training for more epochs on a smaller (10%) dataset often yielded better validation performance than training for a single epoch on the full dataset, suggesting significant data redundancy. It was observed that there were no *significant* differences while training with more epochs. The evaluations also back up this observation.

5.1.1 Model Evaluation Table

The table below summarizes the performance of small models trained on TinyStories.

Hidden Size	Layers	Heads	Eval Loss	Creativity	Grammar	Consistency	Plot Sense
128	8	4	2.00	1.9	2.7	1.8	2.1
128	12	4	1.96	2.1	3.15	2.55	2.15
256	8	2	1.60	2.4	4.8	4.55	3.45
256	8	4	1.59	2.15	3.85	3.7	2.65
256	8	8	1.58	2.5	5.6	5.75	3.5
256	12	4	1.55	2.25	4.9	4.2	2.8
512	8	4	1.33	2.6	5.85	4.7	3.65
512	12	4	1.30	2.55	6.0	5.8	4.2

Table 1: Evaluation of Model Configurations on TinyStories Dataset

5.1.2 Variation across hidden size

Figure 9: We can see that hidden dimension has significant effect on the model performance

5.1.3 Variation across number of layers

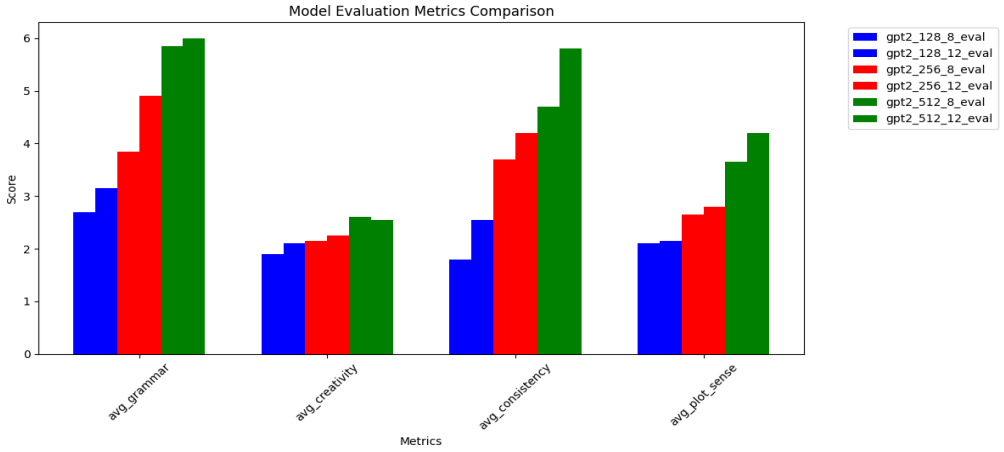


Figure 10: We can see that number of layers also has a significant effect on the model performance

5.1.4 Variation across number of heads

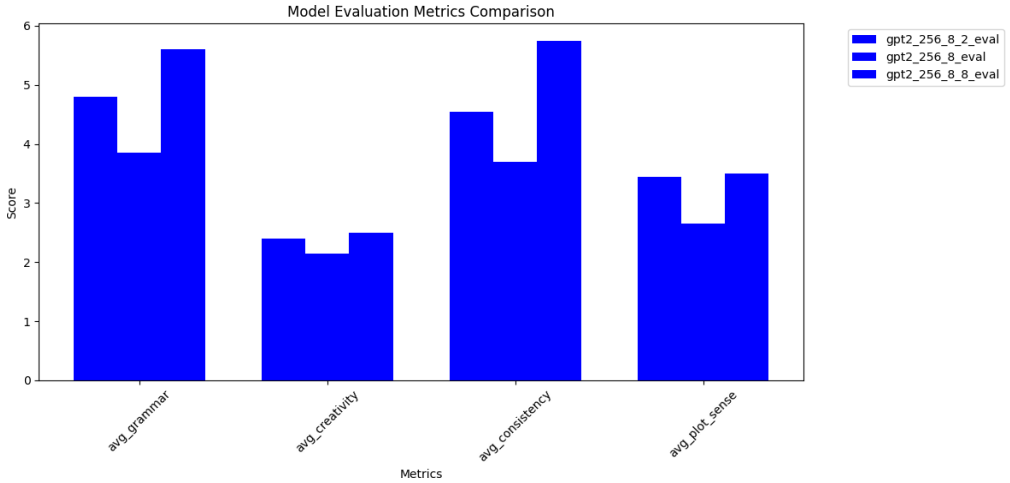


Figure 11: We can see that number of heads doesn't have a significant effect on the model performance

5.1.5 Comparison with pre-trained LLMs

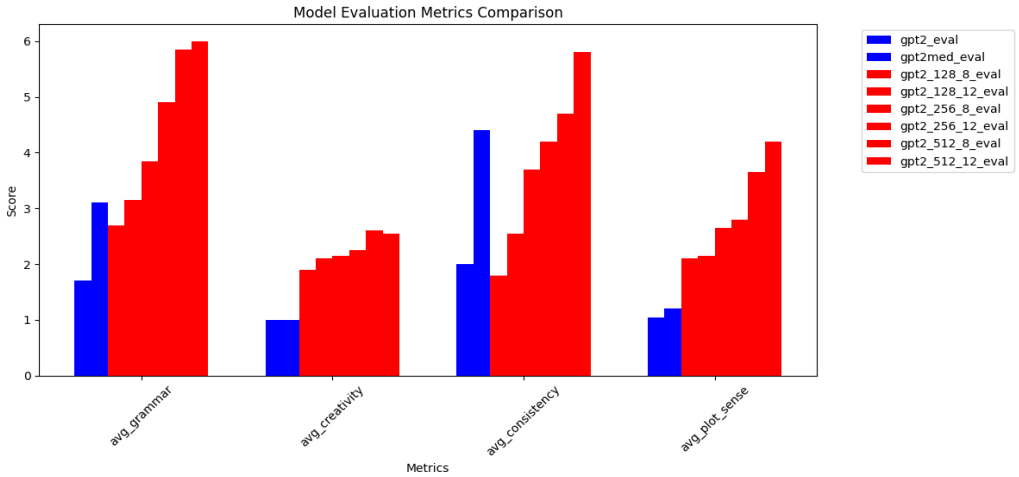


Figure 12: We can see that despite having fraction of parameters, SLMs surpass LLMs in short story generation

5.1.6 Knowledge, Reasoning and Context Tracking

Factual prompts are designed to test a model's knowledge of commonsense facts, ensuring it can accurately recall and apply basic information. Reasoning prompts, on the other hand, assess the model's ability to perform fundamental reasoning tasks, such as understanding cause and effect or making logical eliminations. Additionally, consistency, or context-tracking, prompts evaluate the model's capability to maintain coherence and continuity within a given context, including accurately tracking character names, actions, settings, and plot details throughout a narrative.

Message	256-8-4	512-8-4	256-8-8	512-12-4	contrast-256	contrast-512
Both Ben and Lily wanted cake. Father said that there was only one piece of cake left. They	said that they were good for their birthday.	said that they should share and be kind.	said they could have a cake.	said they had to wait until the cake was ready.	said it was a cake for them.	said that they should not have to share.
On weekends Jack went to visit his grandmother whereas on weekdays he would go to school. Last weekend, when Jack was on his way to	school, he was very excited to see all the different things he had.	school, he saw a big black- board.	school, he saw a big, scary monster.	school, he was excited to see all the new things he could do.	school, he saw a big, red car.	school, he saw his friend Sam.
Jack's mother was not home, and his father was at home. When Jack came home, he said hello to	his parents	his father.	his mother and said, ¨I'm here to visit me.	his mother.	his father.	his father.

Table 2: Reasoning Evals

Message	256-8-4	512-8-4	256-8-8	512-12-4	contrast-256	contrast-512
Alice was so tired when she got back home so she went	to bed	to bed.	to bed.	to the kitchen to get some food.	to the park.	to the hospital
Jack wanted to read a book, so he went to	the park.	the store.	the park	the library	the store	the library
Jack and Lily saw a rainbow after a rainy day. They were amazed by the colors. Jack said, "Look, Lily. A rainbow has	a big yellow flower	a rainbow in the sky	Many colors and shapes.	a rainbow.	a rainbow in the sky	many colors

Table 3: Factual Evals

Message	256-8-4	512-8-4	256-8-8	512-12-4	contrast-256	contrast-512
Hi Jane, have you seen Alice? I can't find her anywhere", said Jack.	Jane was very excited.	I can't believe it.	Jack said," Let's go to the park."	She was very happy to see her.	Jane said," I'm sorry, I will find you.	Jane was very excited.
Max had two dogs. One was white and the other was black. Max walked up the street and saw a kid with a dog. He told the kid, "I see you have a Brown dog. I also have	a dog and a dog.	a dog.	a big dog.	a dog.	a dog!" Max was very happy.	a dog.
Anne had a piece of candy in her left pocket and a piece of chocolate in her right pocket. Anne's mom asked her, "Anne, what is that you have in your left pocket?"	it is a special treat	a candy.	I'm sorry, I just wanted to share my candy with you.	it's a special piece of candy.	Mom said, that's a great idea, I will take a bite of your candy	please have some candy in my pocket."

Table 4: Context Tracking Evals

5.1.7 Quantitative measurement of similarity using Rouge score

5.1.7.1 Similarity between Generations and Original Stories

The histograms below show the ROUGE-2 score evaluation for the different models between stories generated by the models with the original story from the dataset. Seed stories are generated by taking some random stories from the train split of the dataset and taking only about first 40% of it and asking the model to fill in the rest to give the generated story. This analysis confirms that the small language models are actually learning the language and merely not memorizing the stories from the training set.

This is an important evaluation to do to confirm generalizing capability of the trained models and have further deeper insights into their working.

5.1.7.2 Similarity across generations

In the below graphs we calculate the max rouge scores across 100 generations. The result is plotted in the histogram below. The low rouge score indicates that the generations are diverse and depend on the seed text and are not repeated across generations.

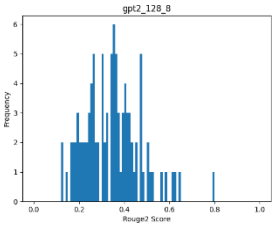


Figure 13.1: ROUGE-2 Score for 128 8

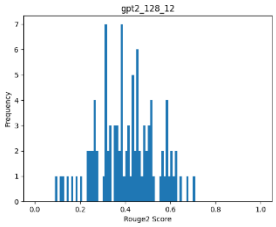


Figure 13.2: ROUGE-2 Score for 128 12

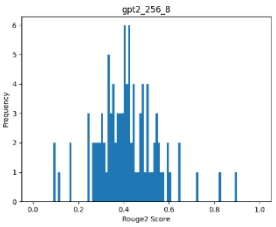


Figure 13.3: ROUGE-2 Score for 256 8

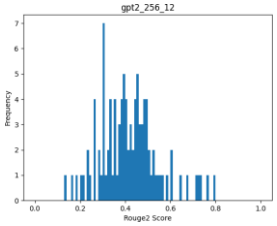


Figure 13.4: ROUGE-2 Score for 256 12

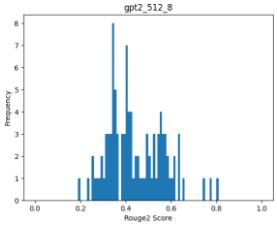


Figure 13.5: ROUGE-2 Score for 512 8

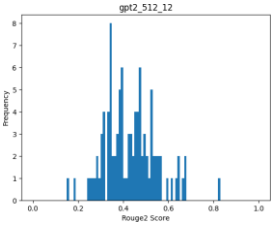


Figure 13.6: ROUGE-2 Score for 512 12

Figure 13: ROUGE-2 Score Evaluation for Different Models

5.2 Architectural Comparison: Transformer vs. Linear Model

In this section, we present the results of our experiments with different configurations of transformer and linear models. Below are the configurations we used for training and evaluation.

5.2.1 Transformer Configurations

We evaluate the following transformer configurations:

Model Name	Vocab Size	dmodel	Num Encoder Layers	Nheads	Epochs
TR-MODEL-1	8192	512	6	8	10 (10% Data)
TR-MODEL-2	8192	512	6	8	1 (100% Data)
TR-MODEL-3	8192	256	6	8	10 (10% Data)
TR-MODEL-4	8192	768	12	8	1 (100% Data)

Table 5: Transformer Configurations

5.2.2 Linear Model Configurations

The configurations used for training the linear model are as follows:

Model Name	Learning Rate	Weight Decay	Epochs	Context Length
LIN-MODEL-1	0.0005	0.1	1	64
LIN-MODEL-2	0.0005	0.1	2	64

Table 6: Linear Model Configurations

5.2.3 Parameter Count

Model Name	Number of Parameters
tr-model-1	27.313M
tr-model-3	12.093M
tr-model-4	78.76M
lin-model-1	775M

Table 7: Model Names and Their Corresponding Number of Parameters

5.2.4 Loss Plots

To analyze the performance of both the transformer and linear models, we present the following loss plots:

Training Loss

Below are the training loss plots for each configuration, presented one after the other:

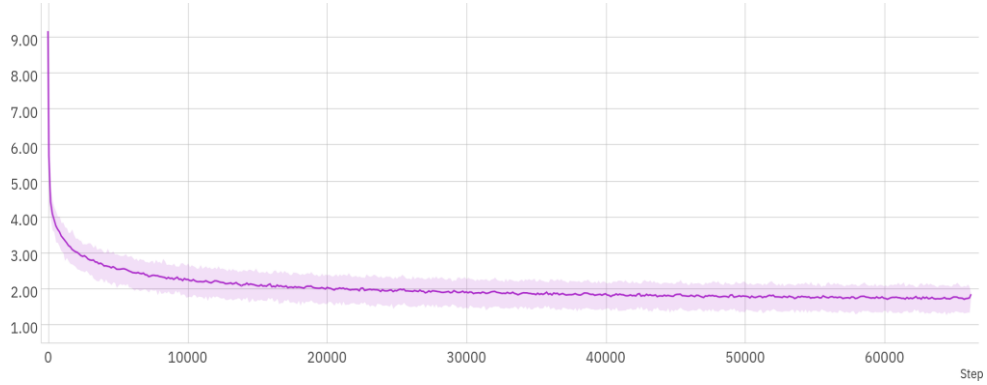


Figure 14: Training Loss for TR-MODEL-1

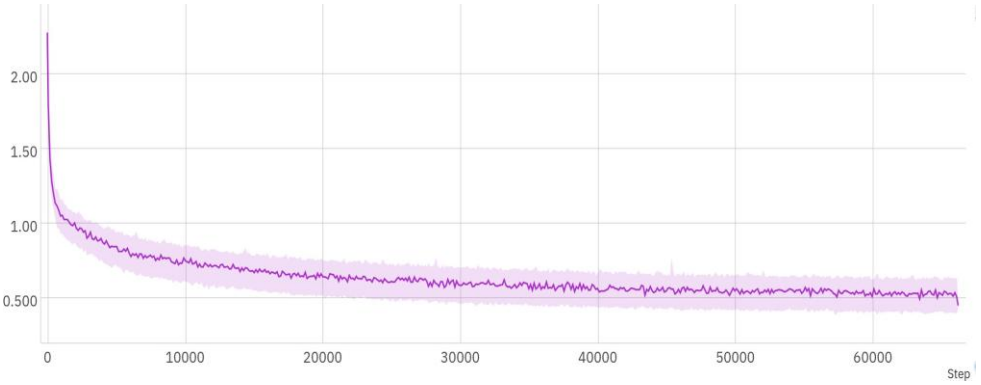


Figure 15: Training Loss for TR-MODEL-2

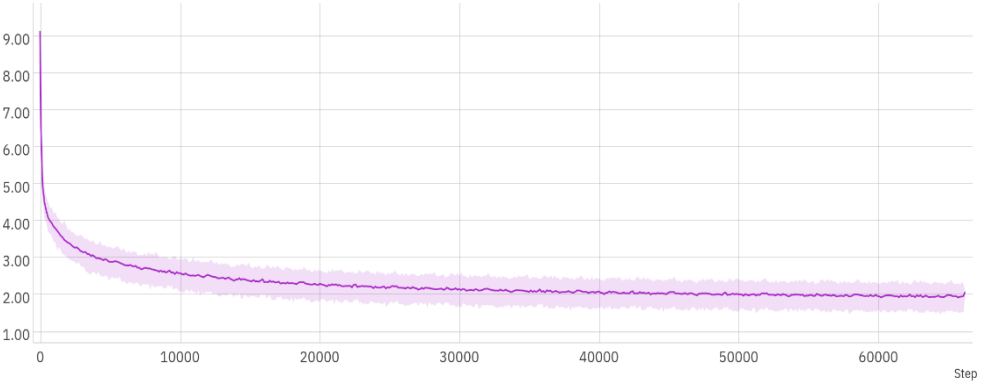


Figure 16: Training Loss for TR-MODEL-3

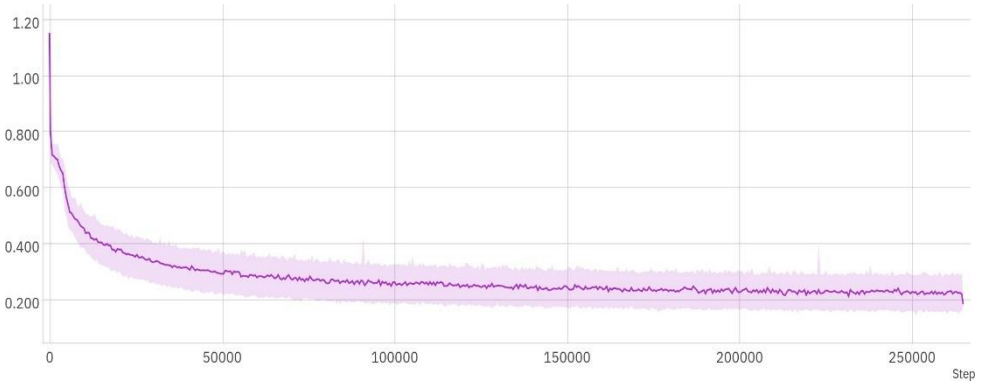


Figure 17: Training Loss for TR-MODEL-4

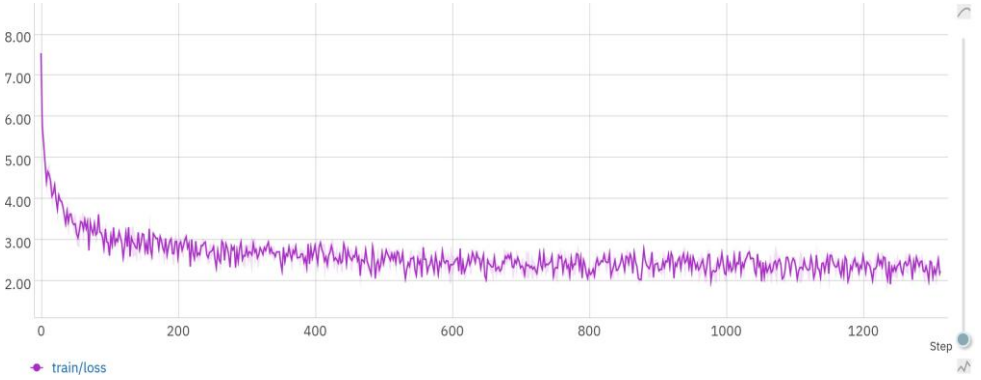


Figure 18: Training Loss for Linear Model (1 Epoch)

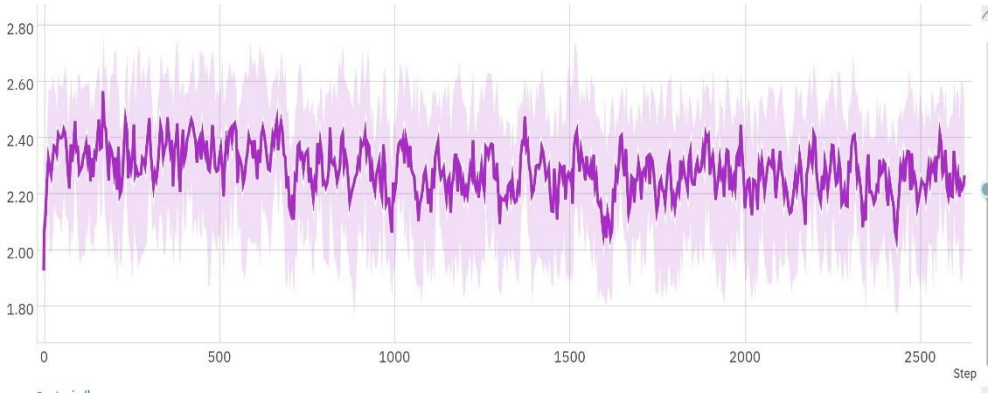


Figure 19: Training Loss for Linear Model (2 Epochs)

Evaluation Loss

Below are the evaluation loss plots for each configuration, presented one after the other:

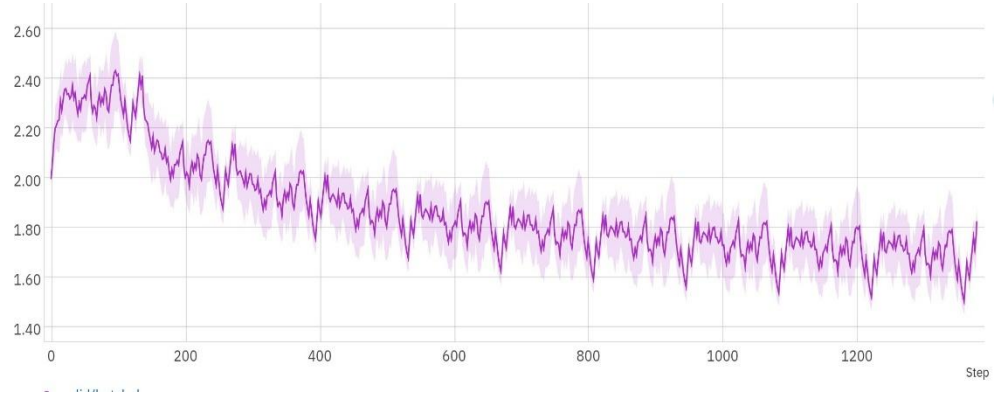


Figure 20: Evaluation Loss for TR-MODEL-1

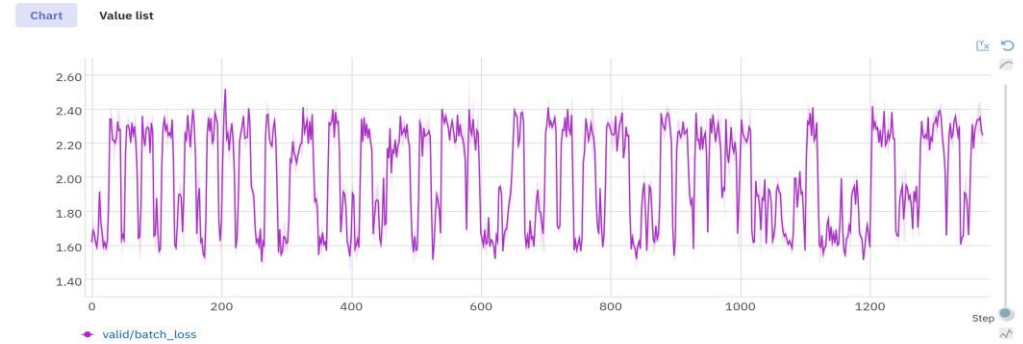


Figure 21: Evaluation Loss for TR-MODEL-2

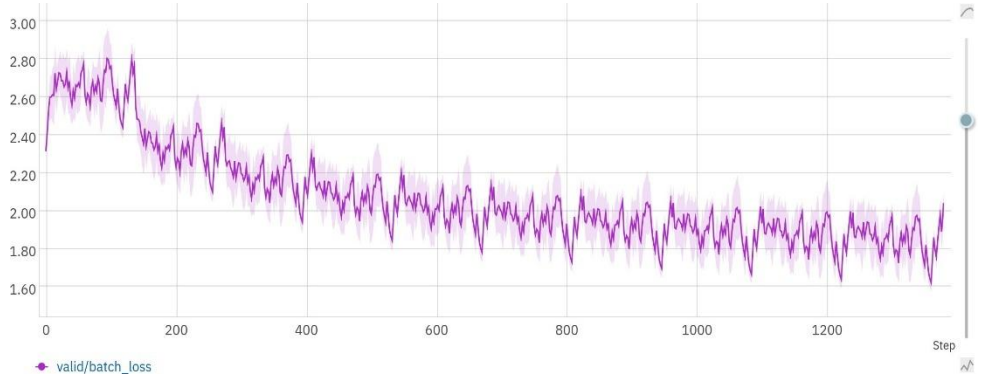


Figure 22: Evaluation Loss for TR-MODEL-3

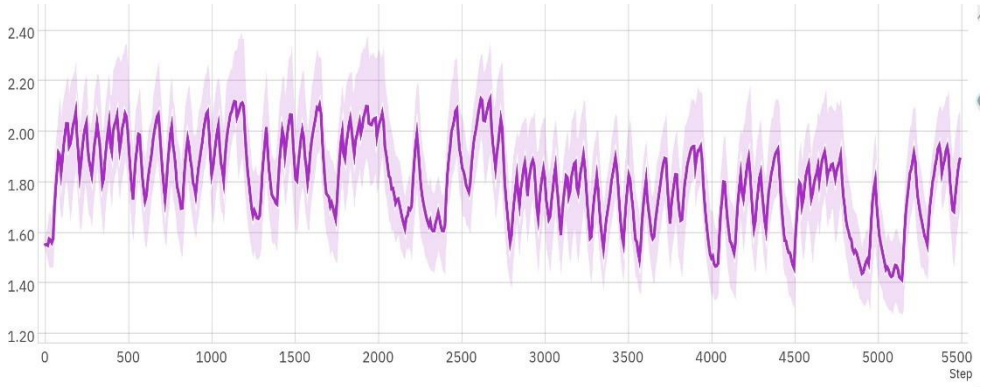


Figure 23: Evaluation Loss for TR-MODEL-4

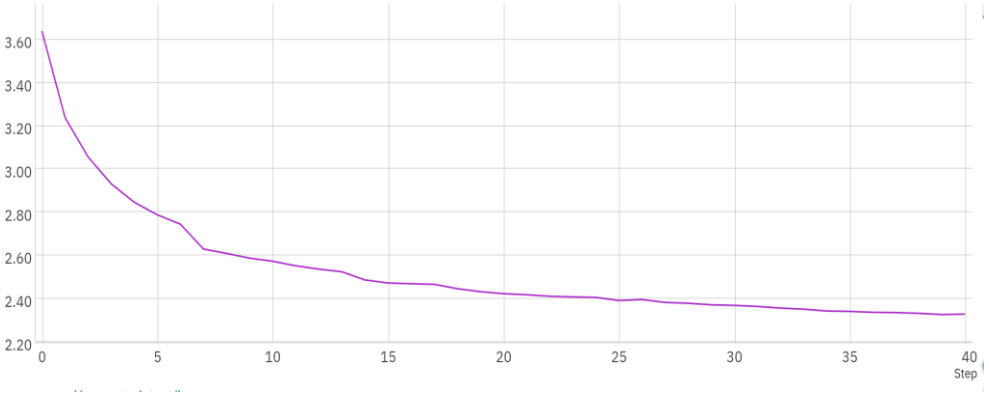


Figure 24: Evaluation Loss for Linear Model (1 Epoch)

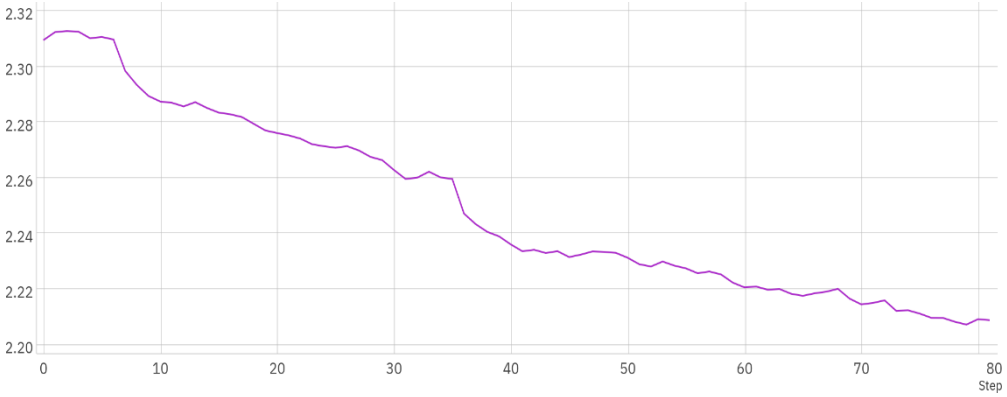


Figure 25: Evaluation Loss for Linear Model (2 Epochs)

5.2.5 Analysis:

1. TR-MODEL-1: The model with a large dimensionality (dmodel = 512) and 6 encoder layers, trained on 10% of the data for 10 epochs, should be able to effectively learn the patterns in the data and show a steep decrease in both training and evaluation loss.
2. TR-MODEL-2: The model with the same dimensionality (dmodel = 512) and encoder layers as TR-MODEL-1, but trained on 100% of the data for only 1 epoch, may struggle to fully learn the

complex patterns in the data, resulting in a slower decrease in both training and evaluation loss compared to TR-MODEL-1. Here we see that evaluation loss oscillates. This potentially could be due to only training it for 1 epoch.

3. TR-MODEL-3: The model with a smaller dimensionality (dmodel = 256) and the same number of encoder layers as TR-MODEL-1, trained on 10% of the data for 10 epochs, may have more difficulty learning the intricate patterns in the data, leading to a slower decrease in training and evaluation loss compared to TR-MODEL-1. Here we do see loss decreasing in both evaluation and training, which is impressive considering the model is of size 12M parameters.
4. TR-MODEL-4: The model with the largest dimensionality (dmodel = 768) and 12 encoder layers, trained on 100% of the data for only 1 epoch, may be able to quickly learn the general patterns in the data, resulting in a decrease in training loss. Similar to the other model trained only for one epoch, we don't see much of a convergence in the evaluation loss. This tells us that it's better to see lesser data for longer, rather than more data for a short amount of time. We can also speculate that the distribution of the data is near-uniform.
5. LIN-MODEL-1: Although we do see fluctuations, we see that the training loss decreases. Figure 24 shows the evaluation loss for a linear model trained with 1 epoch, which decreases sharply initially and then gradually flattens out.
6. LIN-MODEL-2: We see almost no changes in the training loss. This could be indicative of the model not learning. This could mean that the model has already identified all necessary patterns in 1 epoch. Figure 25 depicts the evaluation loss for a linear model trained with 2 epochs, which exhibits a similar decreasing trend as the 1 epoch model but with a lower final loss value

5.3 Training Technique Comparison: Baseline vs. Contrastive

5.3.1 Interpretability

Components of a neural network may not have distinct roles and may behave in a complex and messy way. In order to better understand their working, it becomes important to delve into the intricacies of these components to see what exactly is happening. The paper presents some preliminary evidence that training smaller models on TinyStories leads to higher interpretability mainly because of their small size. We particularly focus on the attention heads of the model and the neurons in the MLP. Attention heads exhibit diverse and meaningful functions, such as attending to the previous work, the subject of the sentence, the end of the sentence, or the main topic of the story. We identify the most influential tokens in the MLP for each neuron. Some neurons are found to get activated on words that have a specific role in the sentence.

5.3.2 Interpreting Attention Heads

We fetch the attention patterns from the model output on a particular piece of input. For every head, we generate a heat map from its attention pattern. We then examine the correlation between the token distances and the attention weights. If the correlation is greater than 0.5, we conclude that the attention is positional, else it is semantic. 1The heat maps for gpt2 512 12 model are as:

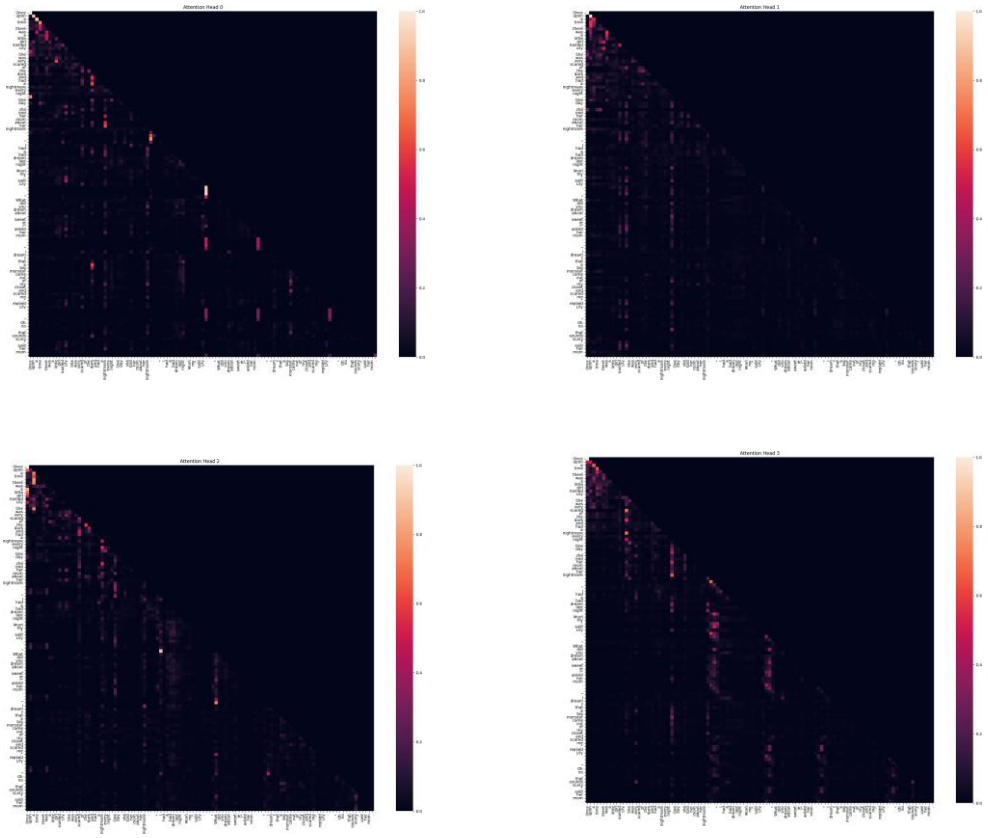


Figure 26: We see that the correlation is very less for all the four heads which implies that all the four heads are involved in providing semantic attention.

5.3.3 Interpreting the roles of neurons

track tokens with the highest activation for each neuron. Below we show results for the gpt2 512 12 model after analysing neurons in some of its layers. We observe that certain neurons in certain layers show higher activations for particular kinds of words. For example, as we show below, the neuron 1 in layer 4 shows higher activations for nouns as compared to verbs, prepositions, adjectives, etc. while the neuron 2 in layer 1 shows higher activations for adjectives as compared to other kinds of words. This shows that the network does get some sense of how the language is constructed.

```
Neuron 1 top activations:
Token:  oats
Context:  found a big bag of oats. He believed that if
Activation: 0.2960
Token:  cartoons
Context:  and laugh at the funny cartoons. But one day,
Activation: 0.2944
Token:  room
Context:  Tom were playing in their room. They liked to watch
Activation: 0.2930
Token:  mud
Context:  loved to play in the mud. He would jump up
Activation: 0.2864
Token:  jungle
Context:  with his friends in the jungle. One day, they
Activation: 0.2825
```

Figure 27: Neuron 1 of layer 4 having highest activations on nouns

```
Neuron 2 top activations:
Token:  videos
Context: . They liked to watch videos on their big TV.
Activation: 0.4705
Token:  graceful
Context:  time, there was a graceful cat named Kitty. She
Activation: 0.4314
Token:  magazines
Context: . He liked to read magazines. One day, he
Activation: 0.3955
Token:  lively
Context:  time, there was a lively little boy named Tim.
Activation: 0.3884
Token:  dull
Context:  boy named Tim found a dull, round rock. He
Activation: 0.3832
Token:  sunny
Context: One sunny day, a little girl
Activation: 0.3410
```

Figure 28: Neuron 2 of layer 1 having high activations on adjectives

5.3.4 Contrastive Training

We calculate the overall loss as: $L = Ln + (\alpha / (Le + 0.01)$

From the above results we see that grammar is learnt much better than creativity and plot sense. Hence, we introduce a new way to train the LM in order to improve these metrics on the same dataset.

During each train loop, we take 33% of the stories and shuffle the sentences in these stories. Along with the Cross Entropy Loss that is normally calculated, Ln, we calculate Contrastive Loss, Lc for the next word prediction of the story with shuffled sentences.

Hence, we try to 'maximize' the loss on the shuffled stories, α is taken as a hyperparameter. We take α=5 for the trainings. 0.01 in the denominator prevents exploding gradient problem.

5.3.5 Results for Contrastive Model

Even though the shuffled stories have proper grammar, we see that the grammar score is not impacted significantly as it learns grammar from the rest of the data. We can see improvements across all other parameters. In all parameters other than grammar, the model with hidden dimension 256 but trained with the contrastive loss method performs on par with the model of dimension 512 trained normally!

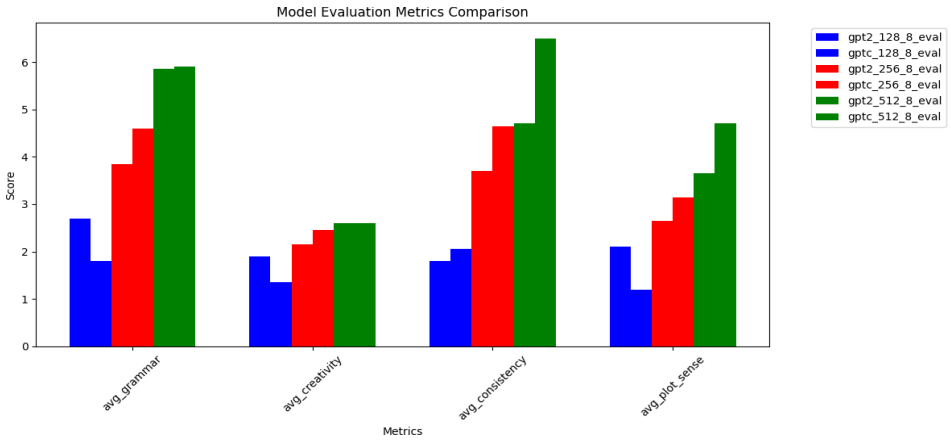


Figure 29: Contrastive Model Evaluations

5.4 Efficiency Comparison: Baseline vs. Compressor

5.4.1 Improving Memory Efficiency

As the model seems to take a large amount of GPU memory, we also tried to improve this memory footprint. A large amount of memory taken by the transformer is a result of the n2 attention matrix that is constructed during the forward step. To resolve this, we use a Encoder only model that takes in 2 tokens at a time, and gives out 1 token, discarding some information but hopefully learning to hold on to the important info. These are then sent to the decoder model that computes the next token in the sequence. These tokens are sent as a source to the decoder model which takes the input sequence Tensor as the target and attempts to get the next two tokens decoded.

5.4.2 Memory Evaluation

	Batch Size 2	Batch Size 4	Batch Size 8	Batch Size 16	Batch Size 32
GPT2					
128	904	2465	4845	9606	oom
256	1076	2930	5691	oom	oom
512	1448	3933	7463	oom	oom
Compressor					
128	457	1272	2361	4517	8831
256	720	1936	3451	6471	oom
512	1351	3408	5781	10529	oom

Table 8: Memory comparison for GPT2 and Compressor across batch sizes. All sizes are in MB

5.4.3 Model Evaluation

Model	Grammar	Creativity	Consistency	Plot Sense
comp-128-8	1.85	1.9	1.5	1.3
comp-256-8	2.6	2.1	2.02	1.6
comp-512-8	3.5	2.21	2.6	2.25

Table 9: Evaluation of models across different metrics

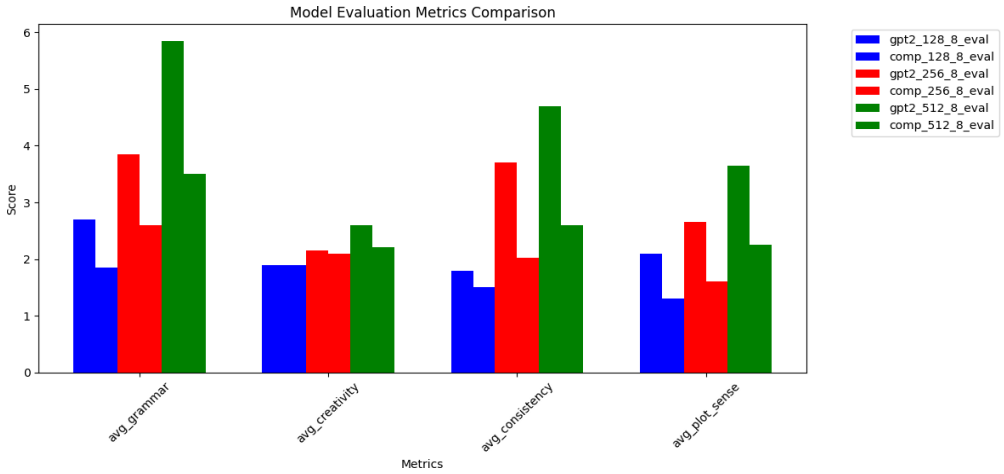


Figure 30: Comparison between Compression and normal model

5.4.4 Loss Curves

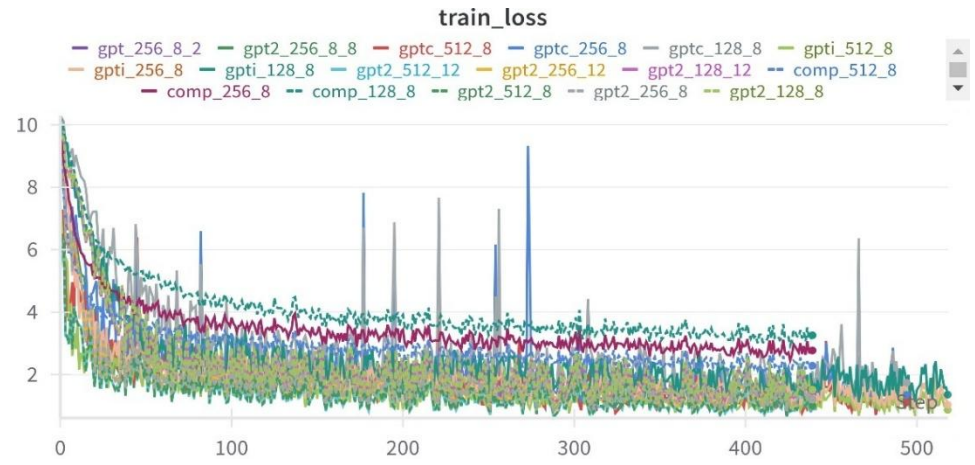


Figure 31: Loss evolution on train set

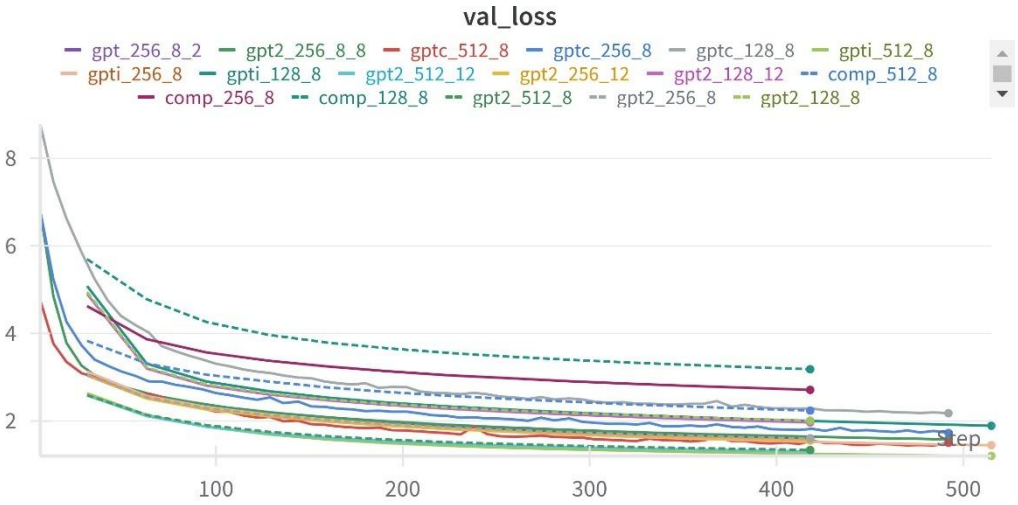


Figure 32: Loss evolution on validation set

We can see some spikes in Loss in Contrastive model training due mostly due to the denominator getting too small. Otherwise, the loss curve is very smooth. We have not trained the model fully due to resource constraints. The model is expected to perform much better if trained for longer.

We can also see that Compression model gets trained the slowest and achieves maximal loss among all the models.

6. Limitations and Future Scope

This project, while comprehensive, has limitations that open avenues for future work.

- **Dataset Cleaning:** A significant effort would be to create a thoroughly cleaned version of the TinyStories dataset and retrain all models to see if performance improves, especially for the simpler architectures.
- **Evaluation Metrics:** The gap between LLM-based and traditional metrics highlights the need for evaluation techniques specifically designed for creative narrative generation.
- **Generalization:** A key unanswered question is how well these specialized models generalize to slightly more complex or out-of-domain data.
- **Model Iteration:** The Compressor and Linear models are promising proofs-of-concept that could be iterated upon to close the performance gap with standard Transformers.

7. Conclusion

Our exploratory analysis of the TinyStories ecosystem yielded several key insights. We confirmed that small, specialized models can achieve remarkable fluency, but revealed that this success is built on a dataset with notable quality issues. The most significant finding is the surprising performance of our Linear Autoregressive Model, which demonstrates that on a simplified domain like TinyStories, the **autoregressive learning paradigm is a more powerful driver of coherence than the architectural complexity of the Transformer**. We further showed that targeted training methods, like our contrastive loss, can yield greater improvements in narrative quality than simply adding parameters. Finally, our dual-evaluation approach underscores the ongoing challenge in NLP: objectively measuring subjective qualities like creativity remains an open problem. This work provides a more nuanced understanding of the factors that enable SLM success and offers a roadmap for building more efficient and effective models for specialized domains

8. References

1. Clark, K., Khandelwal, U., Levy, O., & Manning, C. D. (2019). What Does BERT Look at? An Analysis of BERT's Attention. In T. Linzen, G. Chrupała, Y. Belinkov, & D. Hupkes (Eds.), Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP (pp. 276–286). Association for Computational Linguistics. https://doi.org/10.18653/v1/W19-4828
2. Eldan, R., & Li, Y. (2023). TinyStories: How Small Can Language Models Be and Still Speak Coherent English? arXiv. https://arxiv.org/abs/2305.07759

3. Li, J., Chen, X., Hovy, E., & Jurafsky, D. (2016). Visualizing and Understanding Neural Models in NLP. In K. Knight, A. Nenkova, & O. Rambow (Eds.), Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (pp. 681–691). Association for Computational Linguistics. https://doi.org/10.18653/v1/N16-1082
4. Liu, Z., Zhao, C., Iandola, F., Lai, C., Tian, Y., Fedorov, I., Xiong, Y., Chang, E., Shi, Y., Krishnamoorthi, R., Lai, L., & Chandra, V. (2024). MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. arXiv. https://arxiv.org/abs/2402.14905
5. Vig, J. (2019). A Multiscale Visualization of Attention in the Transformer Model. Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations (pp. 37–42). Association for Computational Linguistics. https://doi.org/10.18653/v1/P19-3007
6. Wan, Z., Wang, X., Liu, C., Alam, S., Zheng, Y., et al. (2023). Efficient Large Language Models: A Survey. arXiv. https://arxiv.org/abs/2312.03863
7. Warstadt, A., Mueller, A., Choshen, L., Wilcox, E., Zhuang, C., Ciro, J., Mosquera, R., Paranjabe, B., Williams, A., Linzen, T., & Cotterell, R. (2023). Findings of the BabyLM Challenge: Sample-Efficient Pretraining on Developmentally Plausible Corpora. In A. Warstadt, A. Mueller, L. Choshen, E. Wilcox, C. Zhuang, J. Ciro, R. Mosquera, B. Paranjabe, A. Williams, T. Linzen, & R. Cotterell (Eds.), Proceedings of the BabyLM Challenge at the 27th Conference on Computational Natural Language Learning (pp. 1–34). Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.conll-babylm.1
8. Wu, R., & Papyan, V. (2024). Linguistic Collapse: Neural Collapse in (Large) Language Models. arXiv. https://arxiv.org/abs/2405.17767
9. Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2024). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. Proceedings of the 37th International Conference on Neural Information Processing Systems.
10. Malach, E. (2023). Auto-Regressive Next-Token Predictors are Universal Learners. arXiv. https://arxiv.org/abs/2310.02227
11. Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). BERTScore: Evaluating Text Generation with BERT. arXiv. https://arxiv.org/abs/1904.09675
12. Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). BLEU: A Method for Automatic Evaluation of Machine Translation. Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (pp. 311–318). https://www.aclweb.org/anthology/P02-1040/